IEEE Open Journal of the Communications Society

Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version XX Month, XXXX.

Digital Object Identifier 10.1109/OJCOMS.XXXX.XXXXXXX

Counting Versus Testing the Destination: A Simulation Study of Layered Defenses Against SMS OTP Flooding and Pumping

NAVEED AHMED[1]

[1]College of Computer and Information Science, Prince Sultan University, Riyadh 11586, Saudi Arabia

CORRESPONDING AUTHOR: Naveed Ahmed (e-mail: nahmed@psu.edu.sa).

This work was supported by Prince Sultan University, which also paid the article processing charge (APC) of this manuscript.

ABSTRACT Services that verify a phone number by SMS expose an endpoint that must be answered before any account exists. Attackers abuse this to waste the service's messaging budget (flooding) or to steer paid messages to premium-rate ranges that share the termination fee with them (SMS pumping). Existing defenses limit the source, score requests with labeled traffic or block prefixes, but no controlled evidence says which destination signal a service should act on, sends per block, or the fate of their codes, or what either costs the users who share a block with an attacker. We evaluate in simulation a twelve-step defense built on app attestation, signed sessions, and verification outcomes against a seven-control baseline, five simpler designs, and 25 attacker profiles, and compare two label-free destination policies, a short-window counter of sends per number block, and sequential tests of code outcomes under two protocols committed before their runs. Attacks with a signature in the request are contained at a low cost to users; a residential attacker is only rationed. Against four concentrated 20-minute pumpers, the counter leaked 6 to 48 messages at 0.06 to 0.43 points of completion, where the default tests leaked 19 to 452; on held-out seeds and workloads, it met its leakage claims but failed its service claims on blocks real users share. Estimates from the rules alone ordered the two policies in 18 of 18 decisive cells. The results rest on simulated users and give operators a tested ordering of destination policies against short, concentrated pumping.

INDEX TERMS Artificially inflated traffic, fraud prevention, one-time password, rate limiting, sequential tests, SMS pumping.

I. INTRODUCTION

MOST online services verify a phone number by sending a code via SMS from an endpoint that must be reachable before any account exists, at a tenth of a cent to twenty cents per message. Two abuses follow from that design. A flooder makes the service send to numbers it does not own, so the service pays for messages nobody asked for; a pumper sends to a range whose operator shares the termination fee, so the service's SMS budget becomes the attacker's income, a technique that MITRE catalogues as resource hijacking [1]. Pumping is the SMS form of international revenue-share fraud, whose voice form was estimated at $10.76 billion a year in 2015 [2]. The sums at a single service are not small: Twitter's owner said in 2023 that pumping had cost the company approximately $60 million a year [3], and Huh *et al.* [4] found as many attack requests as genuine ones in three months of one large service's authentication logs.

Defenses act at three points on the path of a request. At the source, services limit requests per address, session, and device, challenge them with CAPTCHAs, block known proxies, restrict destination countries, and risk-based authentication combines such signals into a score with graded responses [5]. At the messaging vendor, Twilio's Fraud Guard blocks destination prefixes and countries whose traffic departs from an account's history [6], and its per-number

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

pumping risk score advises friction from 60 and refusal above 90 [7]. On a service's own logs, Huh *et al.* [4] detect attack requests one at a time with gradient-boosted trees, at 89.6% recall and 0.19% false positives on web traffic, and Zaliskyi and Odarchenko [8] count application-to-person sends per number prefix in five-minute windows and quarantine a prefix that crosses a threshold.

Each of these rests on an assumption that the literature has tested and found wanting. Source-side limits assume that an identity is scarce; without a trusted authority, one entity can present as many identities as it wants [9]. Addresses are not scarce either: five residential proxy services relayed through 6.2 million addresses in 238 countries, 90% of them for under 15 minutes each and 2.2% of them on any blacklist [10], and carrier-grade NAT, deployed in more than 90% of cellular networks, puts dozens of subscribers behind one address and makes address reputation "all but infeasible" [11]. A CAPTCHA costs an attacker approximately a dollar per thousand solutions from a labor market that was already paying ten dollars per thousand in 2007 [12]. Supervised detection requires labels, which Huh *et al.* [4] derived from traffic peaks after the fact, and a decision taken per request can be probed one threshold at a time. Prefix counting acts on the one key a pumper cannot rotate away from, since it is paid only on its partner's ranges, but the thresholds that stopped 95% and 63% of two incidents in the data of Zaliskyi and Odarchenko [8] also blocked 58% of a legitimate marketing campaign.

Therefore, the two closest studies each act on one destination signal, and neither measures the cost of acting on it. Huh *et al.* [4] report that approximately 92% of attack requests fell on prefixes with zero conversion, against 21% for genuine traffic; thus, whether codes are entered is informative; however, the signal enters a classifier that requires labeled traffic and was not released because it is deployed. Zaliskyi and Odarchenko [8] act on send counts alone, without outcomes, and conclude that counts by themselves cannot safely separate inflated traffic from a campaign. Neither reports what a destination policy costs legitimate users who share a number block with an attacker, with or without an attack, and neither fixed its setting before seeing the evaluation traffic.

A service that sees only its own traffic must decide, for every request, whether to pay for a message to a number it cannot verify, on behalf of a client that has no account yet. Source-side controls tax identities that are cheap to replace, supervised detection needs labels that the service does not have when it decides, and the two destination signals it can compute without labels, the number of sends to a block, and the fate of their codes, have never been compared on the same traffic with the cost to legitimate users made part of the result. The question we address therefore has three parts. On the source side, how much of the flooding that a seven-control baseline admits does a twelve-step redesign contain, at what cost in completed registrations, and through which layer? Where identities are cheap, when does reputation built from verification outcomes stop separating an attacker from the users whose keys it shares, and what does rationing then cost and depend on? On the destination, where a pumper is paid, does counting sends per number block or testing the outcomes of their codes leak less, under which conditions does the answer reverse, and what does either policy cost the legitimate users who share a block, with and without an attack?

We answer these questions in simulation, inside a complete defense for a registration flow, and we release the implementation, protocols, and every run's configuration and outcome [13]. Everything is simulated: three behavioral parameters have published counterparts, and the rest are our assumptions. The main contributions of this study are summarized as follows.

1) We show how an attacker breaks eight assumptions behind a seven-control baseline and close the gaps with a twelve-step pipeline whose trust comes from attestation, signed sessions, and the fate of earlier codes, released as code with a regression test for every defect we found.
2) We measure where reputation built from code outcomes stops separating a residential attacker from the users whose keys it shares, and what rationing it then costs.
3) We compare a short-window counter of sends per number block with sequential tests of code outcomes, selected at a stated service target, frozen, and tested on held-out seeds and workloads, charging each policy with its leakage, its attack-free cost, and its harm to the users who share a block with the attacker.
4) We derive from the rules alone when counting leaks less than testing, and check the estimates against measurements over attacks of up to an hour.

The remainder of this article is organized as follows. Section II reviews the related work and states the gap. Section III presents the baseline and its gaps, the threat model, the design, and its leakage estimates. Section IV describes the implementation, the simulator, the two protocols, and the metrics. Section V presents the results. Section VI discusses them, sets the design against the benchmark designs and the nearest work, and states the limitations. Section VII concludes the article.

II. RELATED WORK

We group prior work by where it acts on the path of an OTP request, which is shown in Fig. 1. The attacker's bots, working through a botnet or behind residential proxies, open the target website and ask it for a code; the website's service endpoint receives each OTP request and decides whether to send; an SMS vendor routes the message; and a terminating operator delivers it, in a pumping attack to premium-rate numbers that a number provider has placed

with that operator. Two flows run against the direction of the request. The outcome signal, whether the code was entered or not, returns from the website to the service endpoint, and it is what one of our two destination policies uses. The revenue-sharing line, drawn dashed, carries part of the termination fee from the operator through the provider of the premium-rate numbers back to the attacker, which is why the destination is the one key a pumper cannot change. The works we review sit at one of these points. At the client, where the bots and proxies sit, are the measurements of what an attacker can buy: residential proxy addresses [10], shared addresses behind carrier-grade NAT [11], CAPTCHA solutions [12], [14], and as many identities as it likes [9]. At the service are the controls an endpoint can run itself: per-entity limits, CAPTCHA, proxy, and country lists (our v1 baseline, Section III-A), risk-based scoring of logins [5], and supervised per-request detection on labeled logs [4]; our twelve source-side steps and two destination policies act here too (Sections III-C–III-E). At the vendor are prefix and country blocking from an account's history and a per-number pumping risk score [6], [7], and counts of sends per prefix in five-minute windows with a 24-hour quarantine [8]. At the operator and the number provider, where the money is made, are the revenue-share number ranges that providers advertise and share, and detection from call records and test-number intelligence [2], [15]. We close with the gap we address (Table 1).

A. REVENUE-SHARE FRAUD

In international revenue-share fraud, a fraudster obtains "pseudo" premium-rate numbers from websites that pay for traffic and generates calls to them; a transit operator short-stops the calls, keeps the termination fee, and shares it with the fraudster. The taxonomy of Sahin *et al.* [2] sets the scheme among frauds that persist because call routing is opaque, each operator knowing only the next hop, and because operators are reluctant to share pricing, routes, or fraud findings with competitors. Sahin and Francillon [15] then measured the market from the supply side, collecting 3,149,793 distinct test numbers that twelve such providers advertised between January 2016 and November 2019 across 238 destinations. The ranges are a managed asset: 80% of them were shared across providers, which subdivide a hijacked block among themselves, and 70% lay inside valid numbering plans. From this intelligence and from call histories, the authors built a random-forest detector for an operator's call records that reached 98% accuracy at 0.28% false positives, where blocking the two-digit neighborhood of a known test number caught 63%. Both studies concern voice telephony, and the detector needs the test-number intelligence and the call histories of an operator. The lesson for SMS is that the number range, not the calling identity, is what the fraudster must keep, and that a party seeing only its own traffic detects the fraud late. An OTP endpoint is in a weaker position still: it holds neither intelligence nor call histories, only its own sends and whether their codes were entered. That is why the policies of Section III-E are keyed on the 8-digit block and use only those two quantities.

B. DETECTING INFLATED SMS TRAFFIC

Huh *et al.* [4] trained gradient-boosted trees on 9.4 million SMS authentication logs from nineteen countries, reaching 89.6% recall at 0.19% false positives on web traffic and 91.1% at 0.11% on app traffic. The logs came unlabeled from one large service over three months of 2023; the authors labeled attacks inside traffic peaks, as web identities with four or more requests and no successful validation in a day, or app number-and-device pairs never validated, and treated requests outside peaks as genuine. Two models with 18 and 13 features at three levels (the request, the identity's history, and country-wide aggregates over 24 hours) run per request before the message is sent. About 73% of web attacks sent four or fewer requests per identity a day, defeating per-entity limits, and destination signals were the strongest: about 92%

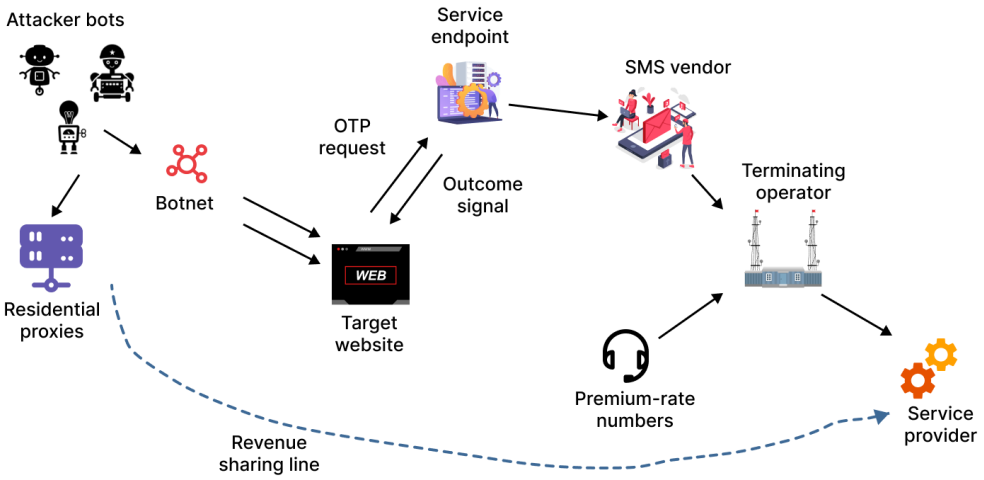


FIGURE 1. The path of an OTP request and where the money goes.

zero conversion for attacks against 21% for genuine traffic, and 91% of app attacks fell on prefixes with more than 30 distinct numbers in a day against 0.01% of genuine requests. The same data show the counter-move: in three countries about 82% of attack requests validated their codes, which the authors attribute to attackers avoiding conversion-rate rules. The detector needs labeled traffic and acts per request, reports its cost to users as false positives only, and was not released because it is deployed. We add a controlled comparison of two label-free ways to act on destination aggregation per block, from send counts or code outcomes, and of their cost to users sharing a block; we did not reproduce theirs, so we do not rank the two.

Zaliskyi and Odarchenko [8] counted A2P sends per prefix (a number less its last three digits) in fixed or rolling five-minute windows and quarantined a prefix for 24 hours past a day or night threshold. On the metadata of two confirmed incidents, fixed windows blocked 95% and 63% of the traffic, and 58% of a legitimate marketing campaign; counting unique recipients blocked 2% of the campaign and none of the first incident. The authors concluded that message counts alone cannot safely separate inflated traffic from a campaign; they did not use verification outcomes. Their numbers, messages blocked on real traffic, are not comparable with ours, which come from simulation and charge a policy with what users sharing a block lose.

The messaging vendors act one hop later. Twilio's Fraud Guard blocks sends to a destination prefix or country whose pattern departs from the account's history, at three protection levels with claimed false-positive rates under 1% and 2% [6]; its SMS Pumping Risk Score rates a number from 0 to 100 from Fraud Guard, risky carriers, traffic patterns and low conversion, advising friction from 60 and refusal above 90 [7]. Graded destination action is established practice, but the signals and models are undisclosed, so we benchmarked no product.

These works establish that attacks concentrate on destination prefixes and avoid converting, that counts alone misfire on campaigns, and that some campaigns now enter codes to pass conversion rules; we modeled the attackers that enter codes, fake receipts, and build trust (Table 2) on that counter-move.

C. WHAT AN ATTACKER CAN BUY

The cost of a source-side control to an attacker has been measured. Motoyama *et al.* [12] bought CAPTCHA solutions from eight retail services at $1 to $20 per thousand, found that the wage for solving had fallen from $10 per thousand in 2007 to about a dollar by 2009, and conclude that a CAPTCHA limits access to attackers whose business is profitable despite that cost rather than keeping machines out. Dionysiou and Athanasopoulos [14] classify the machine-learning solvers that have since appeared. Addresses have a market too: Mi *et al.* [10] infiltrated five residential proxy providers and observed 6.2 million addresses in 238 countries, 95% of them residential, 90% relaying for under 870 seconds and 2.2% on any blacklist; Richter *et al.* [11] found carrier-grade NAT in more than 90% of cellular networks, with dozens of subscribers behind one address, and conclude that address reputation and blacklisting become "all but infeasible" there. Behind both markets lies the older observation of Douceur [9] that without a trusted authority one entity can present many identities, so a signed session provides continuity, not scarcity. Risk-based authentication, which Wiefling *et al.* [5] probed at eight large services, scores a login from its address, user agent, resolution, time, and similar features into risk buckets and asks for a further factor above a threshold; every service that showed such behavior used the address, and none disclosed its procedure. Step 7 of our design resembles it, but scores a request made before any account exists, so the account history that risk-based authentication relies on is absent. Client puzzles [16] would add a computational price per request; however, we did not evaluate them.

These measurements indicate what an attacker can buy. For a defense, they imply that per-identity controls tax identity reset rather than prevent it, and that the question for a residential attacker is not whether it can be told from a person but what rationing it costs the people it resembles. Sections V-C and V-D measure that in this design.

D. SEQUENTIAL TESTS AND ATTACKS ON THE DELIVERED CODE

Our block tests are Wald's sequential tests [17] run as Page's cumulative sums [18], with a floor parameter that selects between the two (Section III-E). Quickest change detection minimizes detection delay under false-alarm constraints [19], and the CUSUM parameters follow from the error rates it must meet [20]; windowed statistics and the time-uniform confidence sequences of Howard *et al.* [21], which stay valid at any stopping time, are not evaluated here; therefore, 'testing' below refers to our conversion and speed tests only, and we report their error rates empirically rather than derive them.

Malware on the handset or a social-engineering call can take a code after it is delivered [22], [23]. These attacks begin where our work stops: we ask whether a message should be sent at all, and leave the protection of a delivered code aside.

E. THE GAP

Table 1 compares the works closest to ours on the evidence they use, the signal they act on, where they act, and whether they measure the cost to legitimate users. Across them, the destination is agreed to be where a pumper is exposed, and three ways to act on it are in use: a supervised model with labels, a count per prefix, and a vendor's undisclosed score. What none supplies is a controlled comparison, under a protocol fixed before the runs, of the two signals a service can compute without labels, with the cost to the users who share a block made part of the result. We supply

that comparison, in simulation, together with the source-side layers it sits in.

III. THREAT MODEL AND DESIGN

This section describes the baseline we replaced, the attacker our design must resist, the twelve steps and two destination policies we designed, and the estimates of leakage we derived from its rules.

A. THE BASELINE AND ITS GAPS

The baseline, v1, is our starting point, the registration endpoint as it stood before this work: it sends a code to any unregistered number once seven controls pass, in the order of Algorithm 1. A request from a known VPN or proxy address (the list $\mathcal{V}$, line 1) is refused. The platform p is read from a client header (line 2), and a request that claims iOS or Android skips the two checks that web requests face, a reCAPTCHA v3 score threshold of 0.5, and an origin check against the allowed domains $\mathcal{O}$ (line 3). The number's country must be on the allow-list $\mathcal{A}$ unless the request carries the bulk flag (line 4); the body must be within the length limit L, and the number not on the exclusion list $\mathcal{X}$ unless the request comes from a test server (line 5). One message per number per minute, five requests per address per minute, and the static caps K per platform and country complete the controls (lines 6–8). Each cap is a counter n that is read when its check runs and incremented after every check has passed (line 9), and the response names the control that refused (line 11).

Each control rests on an assumption that an attacker within the threat model of Section III-B breaks. We derived the eight gaps below from the rules, not from observed abuse, and we give for each the line of Algorithm 1 at which it opens. (A, line 2) The platform exemptions trust a client claim, so any request can send the header for iOS and skip line 3. (B, line 7) Five requests per address assume that addresses are scarce; a residential proxy pool supplies a fresh address per request, nothing throttles a subnet or an autonomous system, and no client identity survives an address change. (C, line 10) A number is sent to without validation, so numbers that cannot exist, numbers on dead ranges and generated sequences cost the same as a real message. (D, line 4) The country allow-list is the only notion of destination cost; premium-rate and revenue-share prefixes inside an allowed country cost many times a standard message, and a pumper targets exactly those. (E, line 8) Every limit is per key, so a wide attack that stays under each cap burns the budget without bound; nothing watches total volume or spend. (F, line 9) A limit checks its counter and increments it as two operations, so two concurrent requests on different instances both pass. (G, lines 4–5) The bulk and test paths skip checks but need no stronger authentication than a normal request, so a leaked flag is a bypass. (H, line 11) Every decision is pass or fail and the response names the control that refused, so an attacker can probe each threshold, stay under it, and enumerate registered numbers.

B. THREAT MODEL

The attacker is a party that wants the service to send as many messages as possible, or as many as possible to numbers that pay it. The first is a flooder: the service pays per message, and the attacker's gain is the service's loss. The second is a pumper, who is paid a share of the termination fee by the operator that terminates its ranges; it earns nothing on any other number, so its traffic concentrates on the blocks it is paid for. The share is not public; therefore, in Section V-H, we report the share at which each scripted pumper breaks

TABLE 1. The closest work is set beside ours.

Work	Evidence	Signal acted on	Where it acts	Cost to legitimate users	Left open for an OTP service
Sahin and Francillon [15]	Operator call records; 3.1 million test numbers advertised by 12 providers, 2016–2019	Distance of the called number to advertised ranges and call history; labels from matched test numbers	Per call, at an operator	A false-positive rate (0.28%), not harm to callers	Voice calls; needs number intelligence and call histories an OTP endpoint does not hold
Huh *et al.* [4]	One service's 9.4 million SMS authentication logs, 19 countries, three months	Per-request features with prefix-level counts and conversion; labels derived from traffic peaks	Per request at the service, before the message is sent	False positives only (0.19% web, 0.11% app)	Needs labels; the deployed model was not released; a per-request decision can be probed
Zaliskyi and Odarchenko [8]	Aggregator delivery metadata: two confirmed incidents and one legitimate campaign	Sends, or unique recipients, per prefix in five-minute windows; no labels	Per prefix, with a 24-hour quarantine	Share of a campaign blocked (58%; 2% with unique recipients)	Outcomes unused; thresholds that stop incidents block campaigns; no per-user cost
Twilio Inc. [6], [7]	Vendor-wide traffic across accounts; documentation only	Departure from an account's history per prefix and country; a per-number score from carriers, patterns and conversion	Before the send, at the vendor; bands at 60, 75 and 90	Claimed false-positive rates under 1% and 2%	Signals and models undisclosed; cannot be benchmarked or reproduced
Wiefling *et al.* [5]	Black-box tests of eight services with 224 accounts	Address, user agent, resolution, time and similar login features scored into risk buckets	Per login to an existing account	Not measured; re-authentication observed	Scores a login, not a request made before any account exists; procedures undisclosed
This study	Simulation: 25 attacker profiles, seven designs, two predeclared protocols; code and every run public	Sends per number block; sequential tests of code outcomes per block; no labels	Per 10,000-number block at the service, before the message is sent	Attack-free benign cost and extra harm under attack, per cohort, in completed registrations	Simulated users and assumed calibration; no real traffic replayed

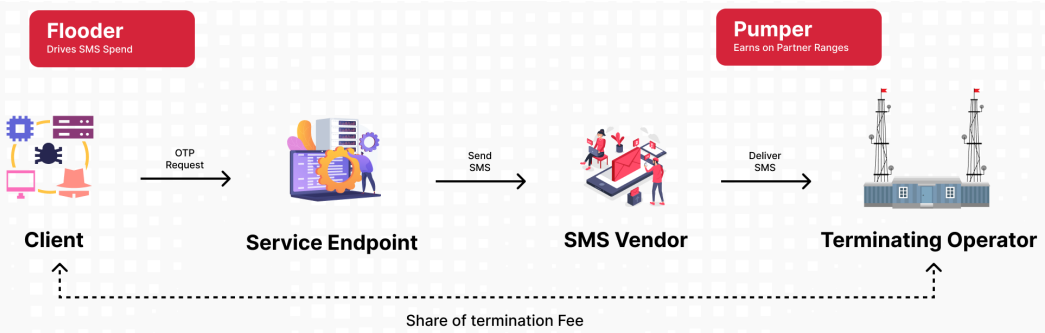


FIGURE 2. **Two attackers on the path and the fee that flows back to one of them.**

Algorithm 1 The v1 baseline.

Require: request r: address, platform header, number, body, bulk and test flags

Ensure: an SMS to $r.\text{number}$, or the name of the control that refused

if $r.\text{address} \in \mathcal{V}$ **then return** refused
$p \leftarrow r.\text{header}[\text{PLATFORM}]$; $u \leftarrow \text{country}(r.\text{number})$
if $p = \text{web}$ **and** $(\text{score}(r) < 0.5$ **or** $\text{origin}(r) \notin \mathcal{O})$ **then return** refused
if $\neg\, r.\text{bulk}$ **and** $u \notin \mathcal{A}$ **then return** refused
if $\neg\, r.\text{test}$ **and** $(|r.\text{body}| > L$ **or** $r.\text{number} \in \mathcal{X})$ **then return** refused
if $n_{\text{num}}[r.\text{number}] \geq 1$ **then return** refused
if $n_{\text{addr}}[r.\text{address}] \geq 5$ **then return** refused
if $n_{\text{cap}}[p, u] \geq K[p, u]$ **then return** refused
increment $n_{\text{num}}[r.\text{number}]$, $n_{\text{addr}}[r.\text{address}]$ and $n_{\text{cap}}[p, u]$
SENDSMS$(r.\text{number}, r.\text{body})$
return sent

even. Fig. 2 places the two attackers on the path of a request: the flooder, whose aim is the service's SMS spend, and the pumper, who earns on its partner's ranges through the share of the termination fee that the dashed line carries back from the operator to the client. What each controls at the four points of the path, what neither can do, and what the model leaves out follow.

At the client, the attacker runs scripts behind residential proxies in the service's own country, with a fresh address per request, or from a datacenter abroad, or spread at a low rate over twenty autonomous systems. It buys human-like reCAPTCHA v3 tokens from solving services at approximately 0.3 cents each; therefore, a score threshold alone does not stop it. It mints a fresh browser fingerprint per request, pre-ages fingerprints for hours, or reuses one two-day-old profile; it can present a forged platform header; and it draws numbers at random, walks them upward, or takes them from its partner's blocks. It knows every threshold, the parameters of the destination tests, and the timing of the workers, and it sees only the uniform response that Section III-C describes.

On its own ranges, a pumper's operator can do what a real recipient can: enter the code it receives, within a second or after a human-like delay, or only when a block's statistic nears its threshold; report deliveries as failed, so that a billed send feeds no test; and verify on hundreds of identity-and-number pairs for ten or thirty minutes to build trusted history before flooding with the same pairs. It can spread over three to three hundred blocks, at full rate or paced to a counter's quota, and it can buy a solution for every interactive challenge.

Three things it cannot do. It cannot forge an app attestation, so a platform established at Step 0 is evidence rather than a claim. It cannot earn on a range its partner does not terminate, which is why the destination is the one key it cannot rotate away from. And it cannot make a stranger enter a code, so a code entered on a block that real users share is honest evidence about that block; what a code entered on the attacker's own number buys it, we measure rather than assume. Attacks on a code after delivery (Section II-D) and compromise of the service, its vendors, or the attestation providers are outside the model. Table 2 lists the 25 profiles that instantiate these capabilities; pool size, rate, and CAPTCHA class are drawn per seed.

C. THE PIPELINE

Fig. 3 shows the implementation as deployed. An HTTP API, run on any number of instances, offers the client five public endpoints: an attestation challenge and an enrolment, a session endpoint where a reCAPTCHA token or an attestation buys a signed session, the request endpoint, which runs the pipeline and returns one uniform body, and the verify endpoint, which takes the code; two internal endpoints, behind a service credential, take the vendor's delivery receipts and drive the timeouts. Behind the API sit the two procedures of Algorithm 2: the pipeline, which runs Steps 0–11 on each request, and the feedback loop, which applies each transition of a send as one compare-and-set. All state lives in one store, Redis in production, and an in-memory twin with the same rules in the tests and the simulator, so what the loop writes the next request reads, which is how the outcome of one send changes the judgment of the next, and any instance can verify any code. One worker per deployment ticks every minute to resolve the sends that timed out and to run the baseline job that sets the multipliers of Step 9 from the pipeline's own counters. Every external

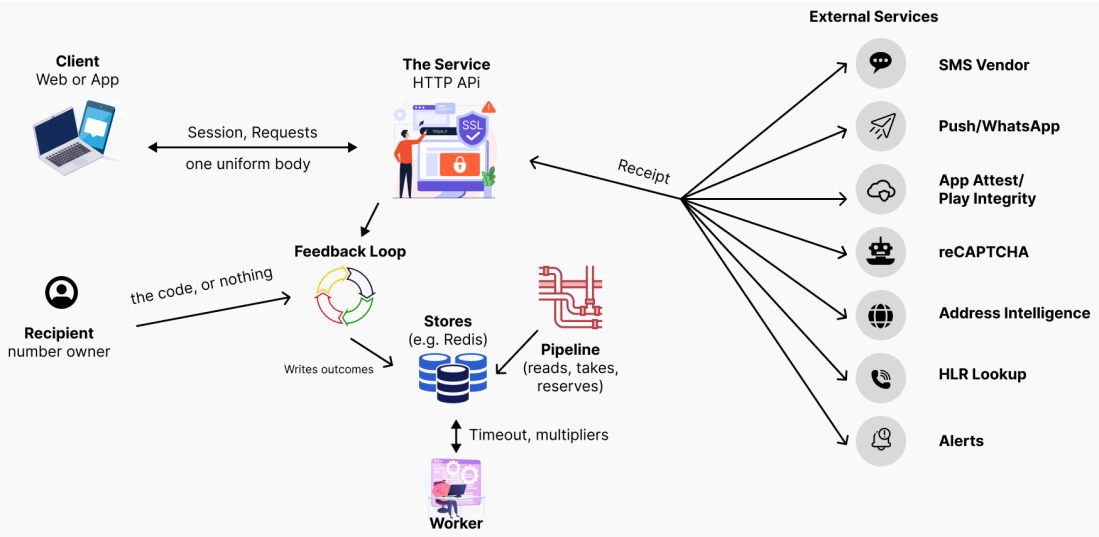


FIGURE 3. How the implementation fits together.

dependency sits behind an adapter with an in-memory fake, and a deployment reports which adapters are fake so that production can refuse to start with any. The simulation of Section IV runs the pipeline, the feedback loop, the worker's two jobs, and the in-memory store as drawn, without the HTTP layer, replaces the external services with their fakes, and in the main study hands the baseline job the modeled legitimate rate instead of letting it learn it (Section V-D runs the learning job).

We designed the pipeline so that no message is paid for before every check passes, and cheap checks run first. Trust comes from evidence that the client cannot forge, an attestation, a server-signed session, and the fate of earlier codes, though such evidence can be manufactured (Section V-F).

A request passes the twelve steps of Fig. 4 in order until one refuses it. The figure numbers its stages in reading order: stages 1 to 4 are Steps 0 to 4, with Steps 3 and 4 drawn as one stage, the web checks; from stage 5 on the numberings coincide, and stage 12 is the feedback loop. The letters below are the gaps of Section III-A that each step closes. Step 0 establishes the platform from an App Attest or Play Integrity result and refuses proxy addresses (A). Step 1 binds the request to a server-signed session with a fingerprint and a replay nonce, three requests per session per ten minutes (B). Step 2 throttles the address, its /24, and its autonomous system, with lower caps for hosting networks (B, F). Steps 3 and 4, for web requests only, check the reCAPTCHA score and the origin. Step 5 prices and checks the number before anything is paid for: its country and cost class, sequential or narrow-range bursts, liveness through an HLR lookup, and the verdict stage of its block, with the bulk flag now needing an internal credential (C, D, G). Step 6 validates the body and the exclusion list, the test flag needing a credential too (G). Step 7 computes the risk score of (1) and chooses one of five tiers (H). Step 8 applies a per-number backoff, sixty seconds doubling to an hour and five sends a day. Step 9 takes the source caps per source, platform, and country, multiplied by the adaptive factor. Step 10 sets the operating mode from the hourly budget, elevated above 80% and emergency at 100% (E). Step 11 reserves the hourly count, spend, and any destination slot in one atomic operation, writes the state, and enqueues the message (E, H). A refusal at any step returns one uniform reply, and every send resolves through the feedback loop, whose per-key and per-block outcomes are what the next request is judged on.

Algorithm 2 provides the same pipeline as that implemented, in the order of the steps, with the feedback loop of Section III-D and the block statistics of Section III-E as its second procedure. Its sets are those of Algorithm 1, and θ_w is the web score threshold. TAKE claims one slot of a limit in one atomic operation and fails once the limit is reached; a refusal at any later line releases every slot claimed so far. CHECKNUMBER returns the number's cost class c, its country u, whether the HLR lookup finds it live, and the verdict stage v of its block; TIER maps the score to one of the five tiers and, for a client without verified history, raises it to challenge if $v = 1$ and to downgrade if $v = 2$; m is the adaptive multiplier of Step 9 and K is the source cap. RESERVE takes the hourly count, the spend, and the block's counter slot together and returns the counter's stage g, by which RAISE lifts the tier of a client without verified history past the quota (Section III-E); DISPATCH sends by SMS, or by push or WhatsApp at the downgrade tier, after a wait at the delay tier; and REPLY is the uniform body. Read against Algorithm 1, the pipeline differs in four ways: the platform is established before anything else and from evidence, every limit is taken atomically and released on refusal, the number is priced and checked before it is paid for, and the outcome of each send flows back into the keys and the destination block that the next request will be judged on. Each limit

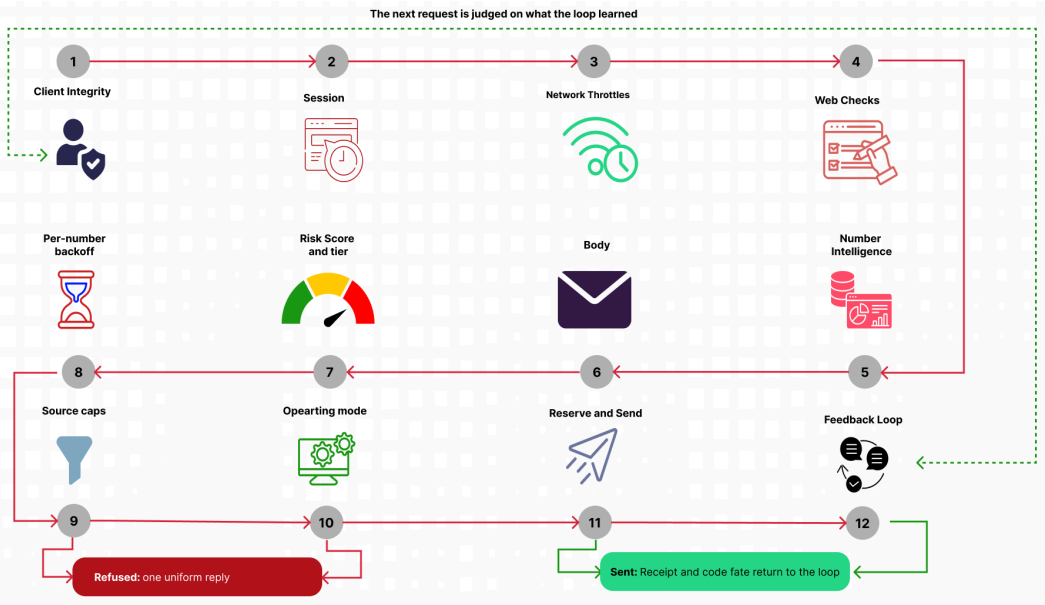


FIGURE 4. **The twelve steps and the loop that feeds the next request.**

and reservation is one atomic operation, but the enclosing steps are not one transaction, and a crash between them can leak reservations (Section IV-A); address limits and source caps count requests, except that a request Step 11 challenges returns its source-cap count. Every outcome after Step 0, except the challenge, returns one HTTP 200 body after at least 400 ms, a floor that cannot hide longer work.

Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint age, session requests, country mismatch, number class), $r_{\min}$ is the lowest conversion ratio over the request's keys (address to destination block), $\theta = 0.3$, $w_c = 25$, f and i flag a flooded key and one whose codes are entered within seconds, and v_n, v_f mark a number and fingerprint with verified history; other weights are 5 to 25 [13]. Tiers start at 20, 40, 60, and 80: *allow*, *delay* (a doubling wait), *challenge* (-20 when solved), *downgrade* (push or WhatsApp), and *block*.

D. THE FEEDBACK LOOP

ONOUTCOME in Algorithm 2 applies one outcome o of a send x. It counts the send on every key it carries, the address, its /24 and ASN, the fingerprint, the session, the country, the prefix and the number, and denylists a fine key for 24 hours once that key has 50 resolved sends and fewer than 10% of them verified; a verification marks the number trusted, which exempts the client from both destination policies. The block's conversion statistic S_c then takes the increment δ_c of Section III-E for the outcome, the speed statistic S_s takes δ_s for a verification, and a statistic that crosses h issues a verdict on the block for an hour and resets both. A send resolves as *verified* when its code is entered, *fails* after five incorrect attempts or two minutes past the delivery receipt, and is *undelivered* without a receipt in 60 s or with a failed receipt; only the first two count toward reputation. A positive receipt after a negative one reopens the send; a late verification reclassifies it, but an issued verdict stands. Each transition records its effects and its time in the same compare-and-set, and a recovery sweep completes the work of a dead process; each effect type applies at most once, and exactly once if the sweep runs within 20 minutes. A block's statistic accumulates in processing order and a verdict is dated at the event that completes the crossing, so a late event can backdate a verdict and move its expiry; simulated runs apply events in time order, but concurrent live instances may not, so the same dependence holds live unless a block's effects are serialized.

E. DESTINATION POLICIES

A pumper is confined to its partner's range but can rotate numbers and blocks within it. Destination policies are keyed on the 8-digit block (10,000 numbers); therefore, protection weakens as the pumper spreads.

1) Sequential Tests

The conversion statistic assumes that users verify with $p_\ell = 0.8$ and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes that a machine enters a verified code within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, because autofill removes the keystroke [24], [25]; it adds

Algorithm 2 The v2 pipeline and its feedback loop.

Require: request r; the store S
Ensure: one uniform reply, after a send, a downgrade, a delay or a refusal

procedure ProcessRequest(r)
  $p \leftarrow$ Attest(r)
  if $p = \perp$ **or** r.address $\in \mathcal{V}$ **then return** Refuse
  if $\neg$ValidSession(r) **or** $n_{\text{sess}}[r.\text{session}] \geq 3$ **then return** Refuse
  if Denylisted(r) **then return** Refuse
  if $\neg$Take(r.address, $r./24$, r.ASN) **then return** Refuse
  if $p =$ web **and** (score(r) $< \theta_w$ **or** origin(r) $\notin \mathcal{O}$) **then return** Refuse
  $(c, u, \text{live}, v) \leftarrow$ CheckNumber(r.number)
  if $\neg$live **or** $c \in \{\text{off-list}, \text{premium}\}$ **then return** Refuse
  if $\neg$ValidBody(r) **or** r.number $\in \mathcal{X}$ **then return** Refuse
  if (r.bulk $\vee$ r.test) $\wedge \neg r$.credential **then return** Refuse
  $s \leftarrow$ the score of (1); $t \leftarrow$ Tier(s, v, r.history)
  if $t =$ block **then return** Refuse
  if $t =$ challenge **then return** Challenge
  if $\neg$Take(backoff[r.number]) **then return** Refuse
  if $\neg$Take($m \cdot K[r.\text{source}, p, u]$) **then return** Refuse
  mode $\leftarrow$ Mode(budget)
  if mode $=$ emergency **then return** Refuse
  (ok, g) $\leftarrow$ Reserve(count, spend, slot[r.block])
  if $\neg$ok **then return** Refuse
  $t \leftarrow$ Raise(t, g, r.history)
  if $t =$ challenge **then return** Challenge
  Dispatch(r, t); Write(S)
  return Reply
end procedure
procedure OnOutcome(x, o)
  for all $k \in$ keys(x) **do**
    $n_k \leftarrow n_k + 1$; $v_k \leftarrow v_k + [o = \text{verified}]$
    if $n_k \geq 50$ **and** $v_k < 0.1\, n_k$ **then** Denylist(k, 24 h)
  end for
  if $o =$ verified **then** trusted[x.number] $\leftarrow$ true
  $b \leftarrow x$.block; $S_c[b] \leftarrow \max(-ch,\ S_c[b] + \delta_c(o))$
  if $o =$ verified **then** $S_s[b] \leftarrow \max(-ch,\ S_s[b] + \delta_s(x))$
  if $S_c[b] > h$ **or** $S_s[b] > h$ **then** Verdict(b, 1 h);
  $S_c[b], S_s[b] \leftarrow 0$
end procedure

1.50 or -2.08. A verdict is issued, and the statistic is reset when either exceeds $h = \ln 1000 = 6.91$. The floor $S_t = \max(-ch,\ S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald's test, $c = 0$ Page's cumulative sum; the default is $c = 1$. A first verdict makes the block's first-time clients pass a challenge for an hour; a second verdict within the hour moves them off SMS, refusing those without another channel.

2) Counters

A counter limits the number of SMS sent per block in a daily or short refilling window. Step 11 reserves a slot and decides the stage in one atomic operation when an SMS is about to be sent, so that concurrent requests cannot skip the challenge. Beyond the limit, a graded counter challenges first-time clients, and past twice the limit moves them off SMS; a refusing counter refuses. Clients with verified history are exempt from both policies; therefore, a code entered on a pumper's number can buy that number an exemption. We also explored a token bucket of the same rate (burst 4 or 12; its challenge-tier bucket refills independently, unlike RFC 2697's [26]) and the counter combined with the tests.

F. CAPACITY CONTROLS

Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job, comparing each key with its expected rate, learned from the pipeline's hourly counters; without history, the static cap stands. Verified history avoids adaptive reduction, not static caps or the budget, which bounds SMS count and spend, not vendor or fallback costs.

G. LEAKAGE ESTIMATES

We compare the SMS sent to a pumper's number over an attack of $T \leq 60$ minutes, by a pumper that solves no challenge and has no verified history. Against the sequential tests, one reaching B blocks with N requests at rate λ leaks about

$$L \approx \min\bigl(N,\ kB + \lambda\,\tau\bigr), \tag{2}$$

before its first verdict, where $k = \lceil h/1.50 \rceil = 5$ and τ, the time a send stays unresolved, is the resolution timeout plus the delivery delay plus half the timeout worker's period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at once. Judged, such a pumper is held for the verdict's hour; therefore, for $T \leq 60$ (2) also estimates the attack's total, ignoring prior credit and outage suspension. A carrier verifying a share σ of codes with a human-like delay gives the conversion statistic an expected increment of $\sigma(-2.08) + (1-\sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it, the statistic drifts away from a verdict.

A counter of q sends per block opens a window with the block's first reserved SMS and closes it after the window length. A pumper that finds its blocks' counters empty (a cold start) sends at most

$$L_c \leq \min\bigl(N,\ qBw\bigr), \qquad w = \lceil T/\text{window} \rceil, \tag{3}$$

whatever its carrier does with the codes. If a legitimate send has already opened a window on block b, using $c_b \geq 1$ slots, the pumper can also take the remaining $(q - c_b)^+$ before at most w further windows open, so in general

$$\begin{aligned} L_c &\leq \min\Bigl(N,\ qBw + \textstyle\sum_{b\ \text{open}} (q - c_b)^+\Bigr) \\ &\leq \min\bigl(N,\ B\,(qw + q - 1)\bigr). \end{aligned} \tag{4}$$

Solving challenges doubles these bounds, and a verified history exempts a client from both policies. While neither

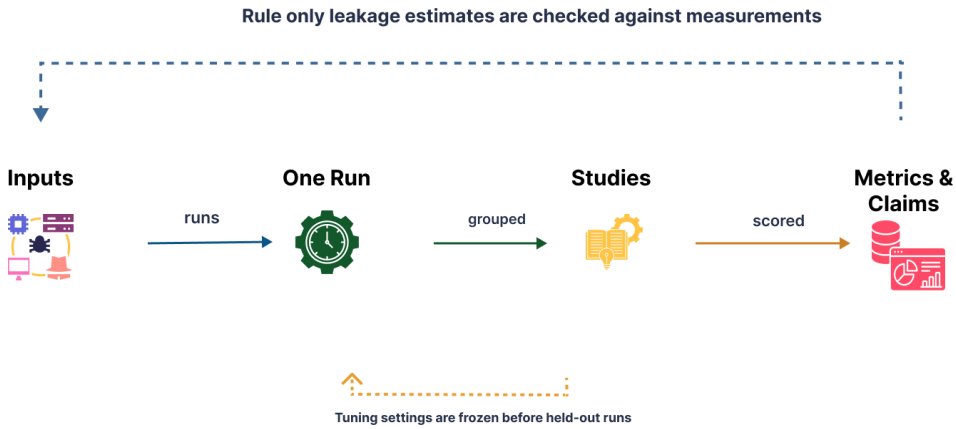


FIGURE 5. **Overview of how we evaluated the design.**

policy's estimate reaches N, the cold-start bound lies below the tests' estimate when

$$B\,(qw - k) < \lambda\tau, \tag{5}$$

and a warm start adds the residual quota to the left side. Equation (5) is the algebraic crossing of two simplified quantities, neither of which is an empirically established switching point or a necessary condition: other limits can hold either policy below its estimate. A carrier that games outcomes may never be judged, so the tests' leakage approaches N.

In the code, every send takes a slot, including the sends of verified clients. If a block's sends arrive as a Poisson stream with mean m per window and all reach the counter, a window holds $1 + X$ of them, $X \sim \mathrm{Poisson}(m)$, and the idealized share beyond the quota is $E[(1+X-q)^{+}]/(1+m)$: 7.3% at $m = q/2$ and 27% at $m = q$ for $q = 4$. This is not a challenge rate: exempt clients take slots without challenge, and challenge failures, the second tier, and earlier gates are ignored. The estimates use only the rules, configuration, and attack rate; what an excess send costs a user needs to be simulated.

IV. METHODOLOGY

Fig. 5 summarizes how we evaluated the design. Our implementation of Algorithm 2 runs inside an event-driven simulator against scripted attackers and simulated users, one run per design, attacker, workload, and seed (Algorithm 3); we group the runs into studies under two protocols that we committed before their runs; and we score each run by what the attacker got through and by what legitimate users lost. The two dashed loops of the figure are the method's two disciplines: settings selected on tuning seeds are frozen before they meet held-out seeds and workloads, and the leakage estimates derived from the rules alone (Section III-G) are checked against the measurements. The rest of this section describes the implementation, simulator, protocols, and metrics in that order.

A. IMPLEMENTATION

We implemented the pipeline of Algorithm 2 in approximately 4,000 lines of Python on in-memory and Redis backends. Up to release 2.8.3, the in-memory store that our simulator uses let a sorted set expire a fixed time after its first write, whereas Redis kept it while written to, so the simulated outage detector and Step 5 lost recent observations. Release 2.9.0 follows Redis in every rule we compared, which differential tests check, and we reran every study with it; in earlier releases we repaired seven replay and concurrency defects, each with a regression test. Of our 453 tests, 19 ran against a real `redis-server` and 18 exercised specific interleavings of two instances, not all. Step 11 is still not one transaction: a crash after its reservations leaks reserved slots or resolves the send as undelivered. The implementation is a research prototype.

B. SIMULATION

We run the pipeline against scripted vendor fakes on a simulated clock, as set out in Algorithm 3. A seed z opens three random streams: for the users, the attacker, and the challenge outcomes; therefore, every scenario at a seed, including the attack-free scenario, offers the same users, and our comparisons between scenarios are paired. The pipeline P is built from the design d on the in-memory store with the vendor fakes, with a fifth of the user pool seeded as returning account holders. Minute by minute, Poisson draws at the legitimate rate λ_ℓ and the attack rate λ_a set the arrivals, the attacker's only from minute 0, and each request q is drawn in full before the pipeline sees it: identity, address, number, CAPTCHA score, delays, and whether the person would verify. A challenge is answered at once when q solves it; a send queues in E a delivery receipt at the queue and delivery delay, and if q verifies, a code entry after its delay; the timed events are applied before each later request and at the end of each minute, when the pipeline's timeouts and its adaptive job also run. After the last minute, the queue is drained through the receipt grace period, resolution timeout, slowest

entry, and a resend. The run record R holds the messages leaked and offered per minute, each user's funnel by cohort, the verdict events, the attacker's verifications, challenges solved and bill, and the configuration with the workload digest and code hash. After a warm-up of ten minutes (30 or 130 for pumping), the attack runs over legitimate traffic at 20 requests per minute. Our simulated users verified 80% of codes after a lognormal delay (median 22 s; 2 s for the 20% using autofill); 70% of numbers were reachable on WhatsApp; 90% of users completed a challenge; 20% were returning account holders with a stable fingerprint, a trusted number, and verified history; therefore, our results for returning users described trusted identities only.

Algorithm 3 One simulation run.

Require: design d, attacker a, legitimate traffic ℓ, seed z, attack minutes T, warm-up minutes W
Ensure: run record R

$(\mathrm{rng}_\ell, \mathrm{rng}_a, \mathrm{rng}_c) \leftarrow$ STREAMS(z)
$P \leftarrow$ BUILDPIPELINE(d); SEEDRETURNINGUSERS$(P, 0.2)$
$E \leftarrow \emptyset$; leaked, offered $\leftarrow \mathbf{0}_T$
for $m \leftarrow -W$ **to** $T-1$ **do**
  $n_\ell \sim \mathrm{Poisson}(\lambda_\ell)$; $n_a \sim \mathrm{Poisson}(\lambda_a)$ $[m \geq 0]$
  $A \leftarrow$ ARRIVALS(n_ℓ, n_a, m) sorted by time
  for all $(t, \mathrm{kind}) \in A$ **do**
    APPLYEVENTS(E, t)
    $q \leftarrow$ DRAWREQUEST(kind); $y \leftarrow P$.PROCESSREQUEST(q)
    if $y =$ challenge **and** q.solves **then**
      $y \leftarrow P$.PROCESSREQUEST$(q \,\|\, \mathrm{solution})$
    end if
    if $y =$ sent **then** $E \leftarrow E \cup \{\mathrm{receipt}(q), \mathrm{entry}(q)\}$
  end for
  APPLYEVENTS$(E, \mathrm{end}(m))$; P.RESOLVETIMEOUTS()
  P.ADAPTIVEJOB()
  if $m \geq 0$ **then** leaked$[m] \leftarrow$ SENTTO(a, m); offered$[m] \leftarrow n_a$
end for
APPLYEVENTS(E, ∞)
return $R \leftarrow$ (leaked, offered, FUNNELS(P), EVENTS(P), d, a, ℓ, z)

Three behavioral parameters have published counterparts: genuine conversion (one provider's 68% or more [27]; ours 60%–90%) and the never-converting destinations and identity churn of Section II; the rest are our assumptions. Table 2 groups the attacker profiles we scripted by capability and marks the destination policies each faced (S, sequential tests; C, counters; B, block-limit; parameters in `results/attacker_profiles.md`). Attackers draw pool size, rate, and CAPTCHA class per seed, which explains 92% [80, 98] of the variance of the farm's leakage in a nested design (ten configurations, three seeds each). Brackets in the text and tables, and bars and bands in the figures, are 95% percentile-bootstrap intervals over seeds, paired per seed for differences; they describe variation under our generator, not uncertainty about real traffic.

C. THE TWO PROTOCOLS

We wrote two protocols and committed each before its runs; a supplementary map (`results/study_map.md`) gives each study's seeds. The first fixed the matched comparison and a robustness study over 12 Latin-hypercube points (conversion 0.6–0.9 and five other behavioral parameters) and a held-out family (60 to 120 attack requests a minute, 50,000 to 500,000 addresses, heterogeneous legitimate traffic), with claims C1–C5 that we count as holding if true in 90% of cells, a benchmark convention, not a confidence level. The matched comparison changes only the destination policy and, at three block densities, selects on tuning seeds the least leakage summed over four concentrated pumpers among settings at or above the *tuning target*, the default's completion minus 0.5 points. The robustness study's first run failed C1 in 32 of 108 cells because attack and no-attack runs drew different users; we fixed the pairing, a fix that followed a result, and reran every study. We wrote the second protocol (claims K1–K5) after the matched comparison had selected three counters and committed it before its runs; six of its seed values had served other robustness points, never selection. We reran both unchanged with 2.9.0 (9,919, 7,925, and 240 runs, each carrying its code hash); against 2.8.0, total leakage changed in 855 of 17,964 comparable runs, but the principal selections, eligibility decisions, and claim counts did not; one auxiliary setting (false alarms matched at unbounded credit) moved from $T1000$ to $T300$ under both protocols (artifact change report).

D. DESIGNS AND METRICS

We evaluated seven designs: **v1**, **v1-corrected** (attested platform), **budget-only** (sessions, static caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. We call a seed *contained* when leakage stays at most 5% of the attack rate from some minute to the window's end, for five minutes or more. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic we call it the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

V. RESULTS

A. WHAT IS CONTAINED

Table 3 lists the leaked share (%, mean of per-run shares) of approximately 600 attack requests in 20 minutes over 30 seeds under v1, block-limit (BL) and v2 with source caps lifted, and under v2 with caps on, with the first-time users refused and all users completing (%); Supplementary Table S1 (`results/design_table.md`) lists every design with intervals. Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the

TABLE 2. The 25 attacker profiles and the destination policies each one faced.

Capability	Profiles	Policies
A signature in the request (5)	One address; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
Games outcomes or trust (8)	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)	S, C
Concentrated on the destination (5)	Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter's quota per block	S, C

TABLE 3. What each design leaks and what users pay for it.

	caps lifted: leaked			caps on, v2		
Attacker	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Aged fingerprints	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

single client to two messages (Table 3); in all 108 robustness cells, these attacks leaked under 5% at completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block's verdict (block-limit: five), and the reused profile 89 [83, 95].

The farm leaks 99.5% with caps lifted under every design; v2's adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so 45.5% complete instead of 76.4% under v1: rationing (claim C5 held in 36 of 36 cells).

B. WHAT EACH LAYER DOES

In the ablation grid (90 effects, each conditional on all other layers, unadjusted intervals; artifact), removing the adaptive caps lets the farm leak 310 [194, 421] more messages; the risk engine, feedback loop, and block key hold the walk by 166, 152, and 121 messages, respectively, and the loop holds the reused profile by 138. The key acts only through the loop (interaction -121 [-141, -101]).

C. WHERE REPUTATION STOPS SEPARATING

A farmed-CAPTCHA attacker uses an address and fingerprint once and shares its country, prefix, and residential ASN with every legitimate user; thus, its failures penalize everyone: over an hour with caps lifted (five seeds), up to twice the legitimate rate, 99 to 100% of it leaks, since a shared key stays above 0.3 conversion until failures outnumber legitimate sends about 1.7 to one. At ten times, 70% leaks while 17% of legitimate users are challenged and 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36 cells). No single score change separates better: a fresh-fingerprint weight of 40 stops it only by challenging 47% of legitimate users. In the settings run, reputation separated an attacker only on keys its sends dominated [9].

D. WHAT RATIONING COSTS

When separation fails, the cap is the lever: over ten seeds, the default lets 47% of the farm's requests through and refuses 41% of all legitimate users, the tightest setting being 20% and 64%. Both assume an oracle job that is told the legitimate rate and runs every minute; running every five minutes or hourly, it lets 75%–96% of the farm's requests through. Learning the rate from the pipeline counters, the deployed job leaked 193 [161, 229] messages with a clean three-week profile, 340 [239, 441] more on a cold start, 106 [62, 151] more with a stale profile, and 45 [10, 77] more with a poisoned one.

E. PUMPING AGAINST THE TESTS

We ran concentrated pumpers on three blocks with caps lifted over ten seeds, about 574 requests in 20 minutes, and recorded the messages leaked, the runs in which the tests contained the attack, the minutes to containment and to the first verdict, and the codes the carrier entered. A carrier that never verified leaked 570 [419, 736] messages under v1, which contained nothing, and 90 [71, 114] under the v2 default, which contained all ten runs at minute 3.5 [3.1, 3.9] after a first verdict at minute 3.0; a carrier that buys every challenge solution leaked 106 [87, 127] and was contained in all ten runs at minute 4.3. The speed test catches a carrier verifying within a second at 19 [17, 21] messages, contained at minute 1.4 after 19 codes entered, while the carrier verifying 60% of its codes with human-like delay was contained in none of the ten 20-minute runs: it entered 272 codes, its first verdict came at minute 11.6 on average and 452 [365, 539] messages leaked; in an hour, 1,067 [839, 1,310] leaked and two of ten runs were contained, at minute 47.0 on average. Claim C3 (containment with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations, leakage follows the blocks touched: with 30 blocks, five to seven runs in ten remain contained.

TABLE 4. Counting against testing at the service target level.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential $T1000$, $c1$ (default)	90	19	452	106	64.12%	0.01	1.64%	6.5
sequential $T300$, $c0$ (selected at tuning target)	88	14	126	98	63.77%	0.37	6.54%	334.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

TABLE 5. The frozen counter on seeds and workloads that it had never seen.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default's plus 0.25 points	93/144 (65%)	23/36	fails

F. COUNTING AGAINST TESTING

1) At a Service Target

Table 4 reports the first protocol at 200 blocks (144 legitimate sends per block a day; tuning seeds 100–102, evaluation seeds 0–9): 20-minute leakage against the never-verifying (NV), instant (IV), human-like (HL), and challenge-buying (CB) carriers and, from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day; the tuning target is 63.95%. A setting is named by its rule, threshold, and credit; therefore, $T300$, $c0$ is the test with threshold 300 and no credit; every setting of the grid is listed in `results/figure1_key.md`. Paired per seed against the default tests, the selected counter leaked 66 [47, 89] fewer messages against the never-verifying carrier, 428 [340, 515] fewer against the human-like one, and 58 [39, 79] fewer against the challenge buyer, and 6 [3, 7] more against the instant verifier; the sequential setting selected at the tuning target leaked 327 [241, 408] fewer against the human-like carrier and 2 to 7 fewer against the other three; the daily counter leaked 44 [3, 84] fewer against the human-like carrier and 229 to 391 more against the rest. The sequential setting selected at the tuning target holds the human-like carrier to 126 messages only by raising 334 false verdict events a day, and misses the target by 0.18 points on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). The selection is not an artifact of the objective: with the full grid and density-specific attacks (E5), and under other objectives on the tuning runs (worst-case leakage, a first-time-loss constraint, each attacker alone, 2,000 random mixes), the rule selects the same counters, except for the instant verifier alone and 3 of the mixes at 200 blocks.

2) On Held-out Traffic

Table 5 covers 12 shifted points of the held-out family, with held-out attack rates and pools, three seeds each and a 20-minute pumper on three blocks real users share; a claim holds if true in 90% of cells, which share workloads and seeds and are not independent, and IV marks the instant verifier's cells. Frozen and run on this family, the counter leaked 33, 34, 33, and 62 messages against the four carriers, where the default leaked 287, 36, 344, and 287, and no policy 451 to 1,857, at a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack claims: 13 of the 17 cells that broke K3 lie at the three points with the most legitimate traffic, where users and pumper compete for one quota; K5 failed in 51 cells over 11 of 12 points. The counters selected for sparser traffic cost 4.7 and 5.8 points here even without an attack: the quota must match the traffic.

Table 6 reports E1 at 200 blocks (60-minute attacks, seeds 300–309): a pumper on three blocks real users share. Leakage is summed over the four carriers' runs of a seed, which can hide a carrier on which a policy loses (per carrier: artifact); harm is net completions lost in points, averaged over the carriers (negative: a gain), with intervals for leakage and the attacked blocks. Against the attacked no-policy arm, the counter cost all users nothing on average and gained the attacked blocks' users three points on average, since without a destination policy, the pumper exhausts their blocks and the hourly budget; with the human-like carrier alone, it cost them 1.9. Against the attack-free operation, every evaluated policy left those users 5 to 9 points worse off on average (2 to 12 per carrier; 8 with none): none shields a block an attacker shares. Counter and tests together, explored outside the frozen claims, leaked least summed over the four carriers but not against the instant verifier (28; $T300$, $c0$: 22). Against a poisoner flooding all 200 blocks for an hour on the same seeds, the tests leaked less than the counters but cost users

TABLE 6. A pumper on blocks that real users share: What leaks and who pays.

Policy	Leaked (SMS)	Extra harm against attacked, no policy: all users (≈1,200)	Extra harm against attacked, no policy: attacked blocks (≈18)	Degradation against attack-free, same policy: all users	Degradation against attack-free, same policy: attacked blocks	Benign cost: all users
none	5,509 [4,409, 6,567]	–	–	0.19	8.0 [5.1, 10.7]	–
sequential, default	1,149 [953, 1,340]	−0.07	−1.0 [−2.8, 0.8]	0.10	7.0 [4.7, 9.1]	0.02
sequential $T300$, $c0$	397 [312, 481]	0.13	0.9 [−1.3, 3.3]	0.12	8.9 [6.0, 11.7]	0.20
counter, 4 per 10 min	403 [386, 418]	−0.05	−3.2 [−5.0, −1.1]	0.07	4.8 [2.9, 6.7]	0.07
counter + sequential	190 [179, 201]	−0.04	−3.1 [−5.1, −0.7]	0.07	4.9 [3.0, 6.8]	0.08
token bucket, burst 4	424 [412, 437]	−0.12	−3.1 [−5.1, −0.7]	0.07	4.9 [2.8, 7.0]	0.00
token bucket, burst 12	545 [531, 559]	−0.11	−2.1 [−4.3, 0.2]	0.08	5.9 [3.3, 8.8]	0.00

more: of 1,976 messages with no policy, $T300$, $c0$ leaked 858 at an extra harm to all users of 2.55 [2.04, 3.07] points, the default tests 1,357 at 1.09 [0.59, 1.59] and the counter 1,662 at 0.72 [0.44, 1.02], and the token buckets spared the poisoner's users only by letting almost everything through. Returning users, all simulated with verified history, lost nothing.

3) The Counter's Boundary

Figs. 6–8 show the counter's operating boundary (E4, seeds 300–304): each point is a mean over five seeds with its 95% interval, lines join the sampled settings without locating a crossover, and the horizontal axes are logarithmic. Fig. 6 plots the attributable loss of the three hot blocks' users, in points of completion, against the legitimate rate on those blocks, from one half to eight sends per block per ten minutes, with 70% of users reachable on WhatsApp, attack-free (hollow markers) and with a never-verifying pumper on the same blocks (filled); the dotted line is the counter's quota of four. Attack-free, the counter costs a block's users nothing at one send per ten minutes, 2.3 points at four and 5.5 at eight, far below the idealized share beyond the quota (27% at four), since most challenged users complete or switch to WhatsApp; without WhatsApp, eight sends cost 13.2, and a 30-minute launch at three times the normal rate costs 16. With a pumper on the same blocks, both policies turn that cost into a gain at up to four sends per ten minutes (the counter 6 to 15 points, the tests 1 to 10), because without a policy the pumper exhausts those blocks and the hourly budget; at eight sends, both cost their users again, the counter 4 points and the tests 9.

Figs. 7 and 8 plot the messages a pumper got through against the number of blocks it spread over, 3 to 300. Fig. 7 covers a 20-minute attack under no policy, the default tests and the selected counter, for a pumper at full rate (solid lines) and one paced to the counter's quota (dashed). At full rate the counter leaked 25 and 81 messages on 3 and 10 blocks where the tests leaked 110 and 144, the two leaked about the same on 30 blocks (235 against 242), and from

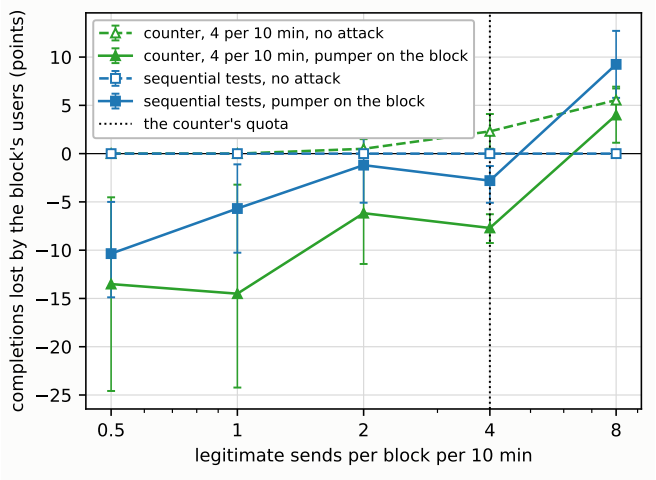


FIGURE 6. What the counter costs the users of a busy block.

100 blocks on both approached the no-policy line of about 700. Paced to the quota, the pumper leaked under the counter what the quota allows, which is what it leaks with no policy (24 messages on three blocks in twenty minutes, 71 in an hour), and a little less under the tests (23).

Fig. 8 shows the hour-long attack at full rate against the estimates of Section III-G, computed from each run's offered requests, offered rate, and configured timeouts, nothing fitted: (2) for the tests and (3) and (4) for the counter's cold- and warm-start bounds, drawn hollow on dotted lines (the 20-minute estimates are in `results/condition_check.json`). Over 60 min, the default leaked 110, 144, and 242 messages on 3, 10, and 30 blocks, against 105, 139, and 240 from (2) ($\lambda\tau \approx 90$), and the counter 71, 240, and 679 against its cold-start bound of 72, 240, and 720; setting aside the exempt sends to real users' trusted numbers, no run that started cold exceeded (3) and none exceeded (4). With these unfitted inputs (5) ordered the policies' mean leakage in 18 of 18 decisive cells (two ties at N), and with the warm-start term in 17, missing where the two estimates differ by 0.8 messages (30 blocks, 20 minutes; measured difference 6.6 [−27.2, 42.4]).

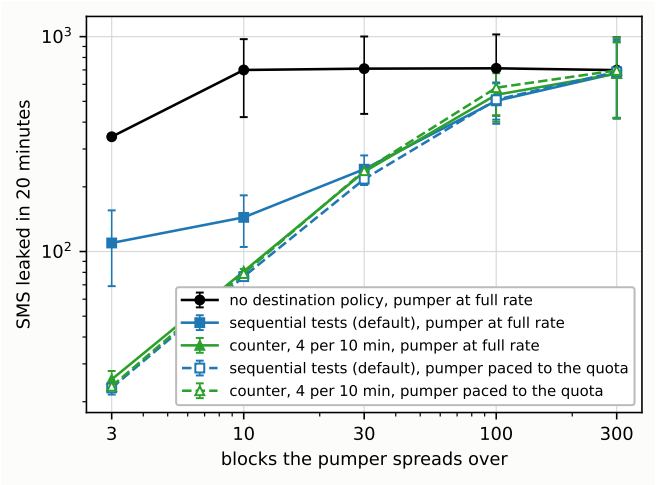


FIGURE 7. **Leakage in 20 min against the blocks a pumper spreads over.**

The algebraic crossing lies near five blocks for an hour and 30 for 20 min, where the nearest measured differences do not exclude zero; it is not a measured switching point.

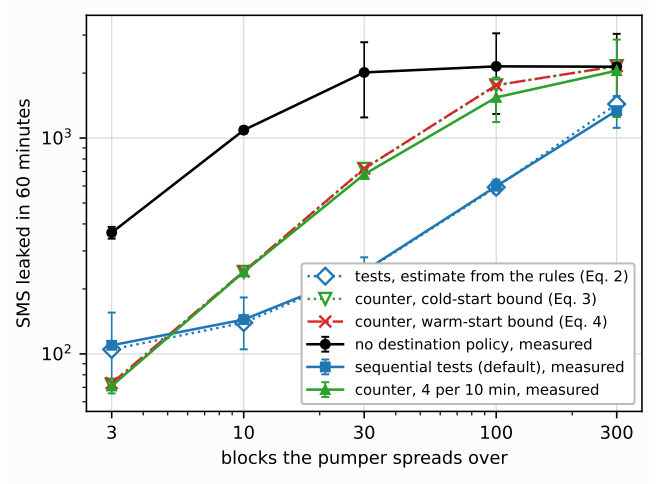


FIGURE 8. **Leakage in 1 h, measured and estimated from the rules.**

Against adaptive attackers (E3), the counter held the threshold-aware carrier, receipt faker, and concentrated trust builder to 24 to 36 messages (502 to 665 under the default), but not trust builders on random numbers (577, as under the default); over 360 minutes, the human-like carrier gained 81 messages an hour under the counter, and 753 under the default. With caps lifted and no counter, a threshold-aware carrier and a receipt faker leaked 570 each, drawing no verdict, and a datacenter attacker that bought every challenge solution 55; the leakages of these scripted adaptations are not upper bounds.

G. THE COST TO LEGITIMATE USERS

On one block, 20,000 Monte Carlo trials per cell give the probability of a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours, the default raises 0.5 verdict events per day on 200 busy blocks at 80% conversion and 6.5 at 65%, where they meet 30 of about 28,800 requests (claim C4 held in 18 of 18 cells).

A poisoner flooding for 20 min the 20 blocks legitimate users concentrate on (caps lifted, ten seeds) raises 16 verdict events meeting 98 legitimate requests, 33 never completing; against observe-only operation on the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of users. One that stops after ten minutes leaves verdicts meeting 406.5 requests afterwards, an attributable loss of 46.2 [16.9, 79.8]. Grading limits the harm; it caps it neither at the challenge nor at the attack duration.

H. ECONOMICS AND OVERHEAD

If pumpers earn a share of the 0.1422 USD retail price and pay an event-level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271 under v2 (the instant verifier 1.310); over 360 minutes under the counter, the human-like carrier needs 1.048. These are break-even shares for scripted attackers under assumed prices, not bounds or criminal profit. On a real Redis (four vCPUs, four workers), sends took 13.8 ms (median) and 21.1 ms (99th percentile), and 133 requests per second passed at concurrency 128 with the 400 ms floor and 50 ms vendor calls; under an adversarial mixture, 81% of sends overran the floor and response-time thresholds separated sends from refusals with 88% balanced accuracy, a separation on that machine, not a deployment accuracy.

VI. DISCUSSION

In the scripted workloads, carriers that verify with human-like delay, fake receipts, or know the thresholds defeated our outcome tests but not a short-window counter: within an hour of a verdict, the counter's allowance, unlike the tests', grows with the windows an attack spans, while over longer campaigns verdicts expire and the tests leak again. Equation (5) summarizes this for pumpers that solve no challenge and hold no exemption; its inputs are in an operator's logs, whereas the cost of an excess send, competition under attack, and exemption abuse come only from the simulation. Supplementary Table S1 sets the seven designs of Section IV against the nine source-side attackers on the same traffic and seeds, with source caps lifted so that only the behavioral layers act; each design adds one thing to the baseline, and its differences from the v1 and v2 columns of Table 3 say what it bought. Correcting the platform header alone changed only the forged-header attack, from 100% to 99.5%, because that attacker is the farm with a header. The behavioral layers of v2 removed datacenter rotation (69% to 0) and most of the reused profile (99.5% to 22.5%), and the two block tests differ only on the walked numbers (11.5% against 13.3%). No design separates the residential bot (37.2%) or the farm (99.5%); those are held only by the adaptive caps of Table 3, at the price Section V-D reports. On the destination, against

the never-verifying pumper v1 leaked 570 messages, the default tests 90 and the selected counter 24, and under the shared-block pumper of Table 6 the counter leaked 403 where the default tests leaked 1,149 and no policy 5,509, neither policy costing all users anything on average.

The nearest external work, set out in Table 1, can be compared only on what each needs and reports, since none ran on the same traffic. Huh *et al.* [4] report 89.6% recall at 0.19% false positives for a supervised model on one service's real logs; the attack it targets, a few requests per identity over many fresh identities, is the farm here, which our layers do not separate and the caps only ration, at 54% of first-time users refused, while what the layers contain without labels are the attacks with a signature in the request and most of the reused profile. Zaliskyi and Odarchenko [8] report that fixed-window prefix counts blocked 95% and 63% of two incidents at the price of 58% of a legitimate campaign; our graded counter is the same family of control, a ten-minute window on a 10,000-number block with a challenge rather than a 24-hour quarantine, and it leaked 24 of about 574 requests from each outcome-reading pumper at an attack-free cost of 0.06 points of completion, on simulated traffic with no campaigns in it, and failed its service claims on blocks the attacker shares. Twilio's products state claimed false-positive rates and report no leakage. What we report that none of the three does is an attack-free benign cost, a cost under attack for the users who share a block with the attacker, a selection fixed before the evaluation runs and tested on held-out seeds and workloads, and an implementation anyone can run; what the three have and we lack is real traffic.

The limitations follow from the method. We simulated everything and assumed the calibration, and we wrote both protocols for our own generator, the second knowing which counters had won. Some results follow from the generator (a perfect premium list, a datacenter attacker that declines challenges, returning users who always present verified history), and our selection rule used an unweighted sum of four scripted pumpers' leakage and a mean service target met on tuning seeds, which guarantees no service level elsewhere. Close rankings rest on ten or fewer seeds, and costs that round to zero are not shown equivalent. Per-client quotas within a block, carrier spend ceilings, bounded exemptions, tuned token buckets, and other sequential statistics lie outside the evaluated policies. Vendor adapters are untested live, and the timing result holds only for the observer tested. Destination policies challenge people for what strangers sharing their block did, and deployment under the GDPR or Saudi Arabia's PDPL needs a lawful basis and a route for refused users.

VII. CONCLUSION

In this work, we asked which of the two label-free destination signals a service that verifies phone numbers by SMS should act on when a pumper is paid at the destination, and we answered it in simulation, inside a twelve-step defense evaluated against a seven-control baseline, five simpler designs, and 25 attacker profiles under two protocols committed before their runs. The design contains the attacks that carry a signature in the request at a low cost to simulated users, while a residential attacker that resembles a person can only be rationed, at a price set by how well the cap tracks the legitimate rate. On the destination, a counter of sends per number block whose quota matches the block's traffic leaked less than tests of code outcomes against short, concentrated pumping and cost its users less, but it failed its service claims on blocks the attacker shares and lost its advantage against paced or widely spread pumpers; estimates derived from the rules alone ordered the two policies in 18 of 18 decisive cells. Every number describes scripted attackers and simulated users under assumed calibration, and close rankings rest on ten or fewer seeds. Replaying real traffic is the experiment this work most needs; the policies left outside the evaluation, per-client quotas within a block, bounded exemptions, tuned token buckets, and time-uniform statistics, could then be compared under the same protocols.

ACKNOWLEDGMENT

The author used Claude (Anthropic), an AI assistant, in preparing this article: to write and review the code of the implementation and the simulation and to run the evaluation (Sections III and IV); to draft and edit the text of every section; to generate Figs. 6–8 and the tables from the recorded results; and to check the references against their publishers' records. Figs. 1–5 are the author's own drawings. The author checked every number against the recorded results and every claim against the code, and takes full responsibility for the content.

DATA AVAILABILITY

Every run's configuration and outcome and the sources of headline numbers (`results/`) are at https://github.com/nahmedpsu/OTP-DDOS, release 2.9.1 (tag v2.9.1), which runs unchanged the code of release 2.9.0 (tag v2.9.0, commit 9af3100) that produced the results (`code_identity.md`); the 2.8.0 results are in `results/historical_2.8`. These files are the supplementary material.

REFERENCES

[1] MITRE Corporation, "Resource hijacking: SMS pumping, technique T1496.003, MITRE ATT&CK," https://attack.mitre.org/techniques/T1496/003/, 2026, accessed 2026-10-01.

[2] M. Sahin, A. Francillon, P. Gupta, and M. Ahamad, "SoK: Fraud in telephony networks," in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. Paris, France: IEEE, 2017, pp. 235–250, doi: 10.1109/EuroSP.2017.40.

[3] Commsrisk, "Elon Musk says Twitter lost $60mn a year because 390 telcos used bot accounts to pump A2P SMS," https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/, 2023, trade press; the figure is the company's own claim.

[4] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, and N. Choi, "Preventing artificially inflated SMS attacks through large-scale traffic inspection," in *34th USENIX Security Symposium*. Seattle, WA, USA: USENIX Association, 2025.

[5] S. Wiefling, L. Lo Iacono, and M. Dürmuth, "Is this really you? An empirical study on risk-based authentication applied in the wild," in *ICT Systems Security and Privacy Protection (IFIP SEC 2019)*, ser. IFIP Advances in Information and Communication Technology, vol. 562. Springer, 2019, pp. 134–148, doi: 10.1007/978-3-030-22312-0_10.
[6] Twilio Inc., "Verify fraud guard," https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard, 2026, accessed 2026-10-01.
[7] Twilio Inc., "SMS pumping risk score (Lookup v2)," https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk, 2026, accessed 2026-10-01.
[8] M. Zaliskyi and M. Odarchenko, "Artificially inflated traffic in A2P messaging: Window-based detection models and prefix-level analysis for fraud mitigation," *Science-Based Technologies*, vol. 69, no. 1, pp. 89–98, 2026, doi: 10.18372/2310-5461.69.20735.
[9] J. R. Douceur, "The Sybil attack," in *Peer-to-Peer Systems (IPTPS 2002)*, ser. LNCS, vol. 2429. Springer, 2002, pp. 251–260, doi: 10.1007/3-540-45748-8_24.
[10] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrwais, L. Sun, and Y. Liu, "Resident evil: Understanding residential IP proxy as a dark service," in *2019 IEEE Symposium on Security and Privacy (S&P)*. San Francisco, CA, USA: IEEE, 2019, doi: 10.1109/SP.2019.00011.
[11] P. Richter, F. Wohlfart, N. Vallina-Rodriguez, M. Allman, R. Bush, A. Feldmann, C. Kreibich, N. Weaver, and V. Paxson, "A multi-perspective analysis of carrier-grade NAT deployment," in *Proceedings of the 2016 ACM Internet Measurement Conference (IMC)*. Santa Monica, CA, USA: ACM, 2016, pp. 215–229, doi: 10.1145/2987443.2987474.
[12] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker, and S. Savage, "Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context," in *19th USENIX Security Symposium*. Washington, DC, USA: USENIX Association, 2010.
[13] N. Ahmed, "OTP Guard: a layered SMS validation, risk-scoring and rate-limiting pipeline against OTP flood abuse (version 2.9.1)," https://github.com/nahmedpsu/OTP-DDOS, tag v2.9.1, 2026, results produced by 2.9.0 (commit 9af3100), whose executable code 2.9.1 keeps; 2.8.0 results (commit 1c277a0) in results/historical_2.8.
[14] A. Dionysiou and E. Athanasopoulos, "SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers," *Computers & Security*, vol. 97, p. 101947, 2020, doi: 10.1016/j.cose.2020.101947.
[15] M. Sahin and A. Francillon, "Understanding and detecting international revenue share fraud," in *Network and Distributed System Security Symposium (NDSS)*. Virtual: Internet Society, 2021, doi: 10.14722/ndss.2021.24051.
[16] D. Dean and A. Stubblefield, "Using client puzzles to protect TLS," in *10th USENIX Security Symposium*. Washington, DC, USA: USENIX Association, 2001.
[17] A. Wald, "Sequential tests of statistical hypotheses," *The Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, 1945, doi: 10.1214/aoms/1177731118.
[18] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954, doi: 10.2307/2333009.
[19] V. V. Veeravalli and T. Banerjee, "Quickest change detection," arXiv:1210.5552 (book chapter), 2012.
[20] NIST/SEMATECH, "CUSUM control charts, section 6.3.2.3, e-handbook of statistical methods," https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm, 2026, accessed 2026-10-02.
[21] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon, "Time-uniform, nonparametric, nonasymptotic confidence sequences," *The Annals of Statistics*, vol. 49, no. 2, pp. 1055–1080, 2021, doi: 10.1214/20-AOS1991.
[22] C. Mulliner, R. Borgaonkar, P. Stewin, and J.-P. Seifert, "SMS-based one-time passwords: Attacks and defense," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2013)*, ser. LNCS, vol. 7967. Springer, 2013, pp. 150–159, doi: 10.1007/978-3-642-39235-1_9.
[23] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, and N. Memon, "Mind your SMSes: Mitigating social engineering in second factor authentication," *Computers & Security*, vol. 65, pp. 14–28, 2017, doi: 10.1016/j.cose.2016.09.009.
[24] Google LLC, "SMS retriever API overview," https://developers.google.com/identity/sms-retriever/overview, 2026, accessed 2026-10-01.
[25] Web Incubator Community Group, "WebOTP API," https://github.com/WICG/WebOTP, 2026, accessed 2026-10-01.
[26] J. Heinanen and R. Guerin, "A single rate three color marker," RFC 2697, Internet Engineering Task Force, 1999, doi: 10.17487/RFC2697.
[27] Twilio Inc., "Verify API," https://www.twilio.com/en-us/user-authentication-identity/verify, 2026, accessed 2026-10-03.

NAVEED AHMED is with the College of Computer and Information Science, Prince Sultan University, Riyadh, Saudi Arabia (e-mail: nahmed@psu.edu.sa).